TFT Screen Library ST7735

Contreras R Orlando[1,2*] and Lara H Kevin[1,2*]

[3]Department of Electronic Systems, UAA, Av. Universidad 940, Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): {al348390,al464376}@edu.uaa.mx;

Abstract

The objective of this TFT_LCD screen driver is implement an easy way to show info on a screen. This is very useful in embedded systems applications, like show images, text, or anything that requires a screen.

Keywords: ARM, C++, Driver, TFT Screen.

1 Introduction

Communication protocols are very useful in a lot of tech applications; all computers, micro controllers, and every circuit with a processor need communication protocols to send or receive I/O data and control external peripherals, some of the most used protocols are SPI. Using this application, the ST7735 TFT screen is a device able to print pixels on a point-matrix, sending via SPI the column and row address to set the internal screen pointer on the pixel that we want to turn on, subsequently we send the color data of this pixel in the color format RGB565.

1.1 RGB565

The RGB565 color format is a 16-bit representation of colors, where 5 bits are used for red, 6 bits for green, and 5 bits for blue. This allows for a total of 65,536 different colors. The format is structured as follows:

Here we got the pink color on RGB565 format (0xF81F), the red color (0xF800), the green color (0x07E0), and the blue color (0x001F), while on RGB888 the pink color is (0xFF00FF), the red color (0xFF0000), the green color (0x00FF00), and the blue color (0x0000FF). Therefore we could get the conversion from RGB888 to RGB565 using the following formula:

$$RGB565 = ((R\&0xF8) << 8)|((G\&0xFC) << 3)|(B >> 3) \tag{1}$$

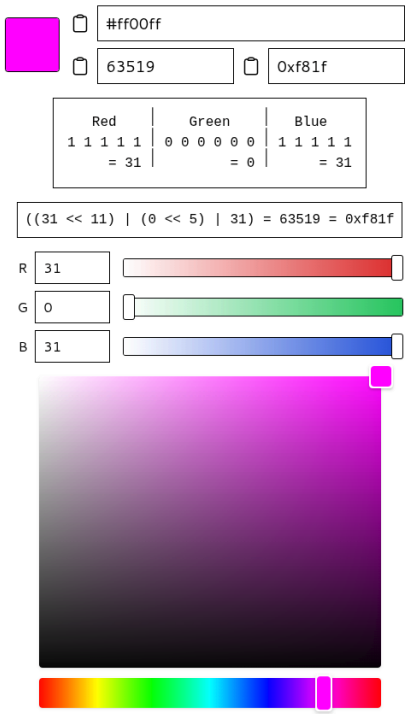


Fig. 1: RGB Format

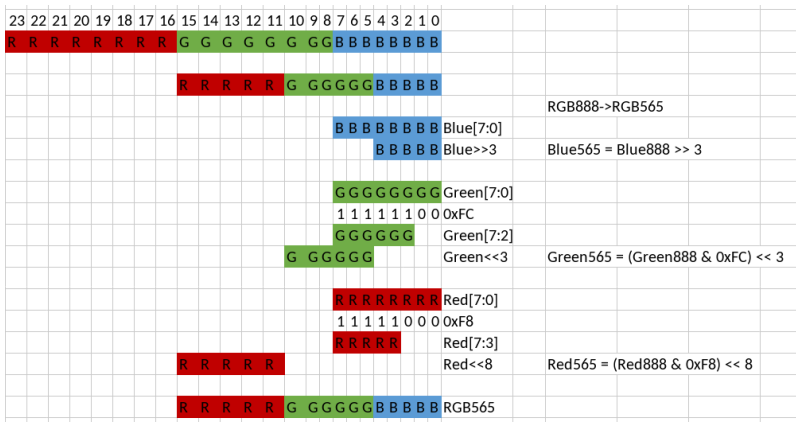


Fig. 2: Conversion

If we didnt had the colors itselfs, we could use the following formula to get the RGB565 color from the RGB888 color:

$$RGB565 = (((RGB888\&0xf80000) >> 8)+((RGB888\&0xfc00) >> 5)+((RGB888\&0xf8) >> 3)); \quad (2)$$

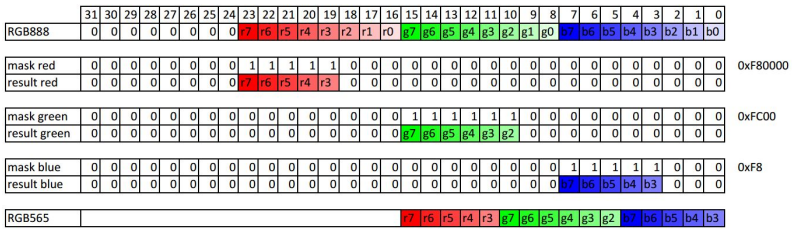


Fig. 3: Enter Caption

2 ST7735

3 ST7735 Interface Architecture

The ST7735 controller is highly versatile and supports both serial and parallel communication interfaces to interact with the host microcontroller. The selection of the interface is typically determined by the hardware configuration of the IM[2:0] pins on the display module. The primary interfaces supported are:

- **Parallel Interface:** 8-bit, 9-bit, 16-bit, or 18-bit 8080-series MCU parallel interface.
- **Serial Interface:** 3-line / 9-bit SPI and 4-line / 8-bit SPI.

3.1 Serial Interface Configurations

The serial interface reduces the pin count drastically. The screen handles two serial modes, controllable via internal settings or configuration pins:

- 3-Line/9-bit Serial interface (CS (Chip Select), SCL (Signal Clock), SDA (Serial Data Input/Output))
- 4-Line/8-bit Serial interface (CS (Chip Select), SCL (Signal Clock), SDA (Serial Data Input/Output), D/CX (Data/Command Flag))

We could switch between them with the $SPI4W$ register configuration:

$$\begin{cases} SPI4W = 0 & \text{(3-Line Serial interface)} \\ SPI4W = 1 & \text{(4-Line Serial interface)} \end{cases}$$

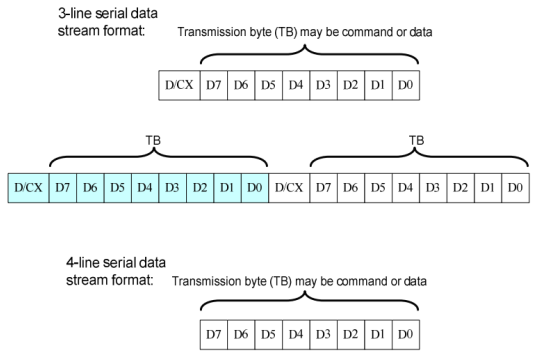


Fig. 4: Serial Interface Timing Diagram

3.2 Time and Voltage Considerations

Should be considered the following specifications for the ST7735 screen, the voltage levels and the time of the signals, as shown in figure 5. We take the VDDI voltage as 3.3V same as VDD.

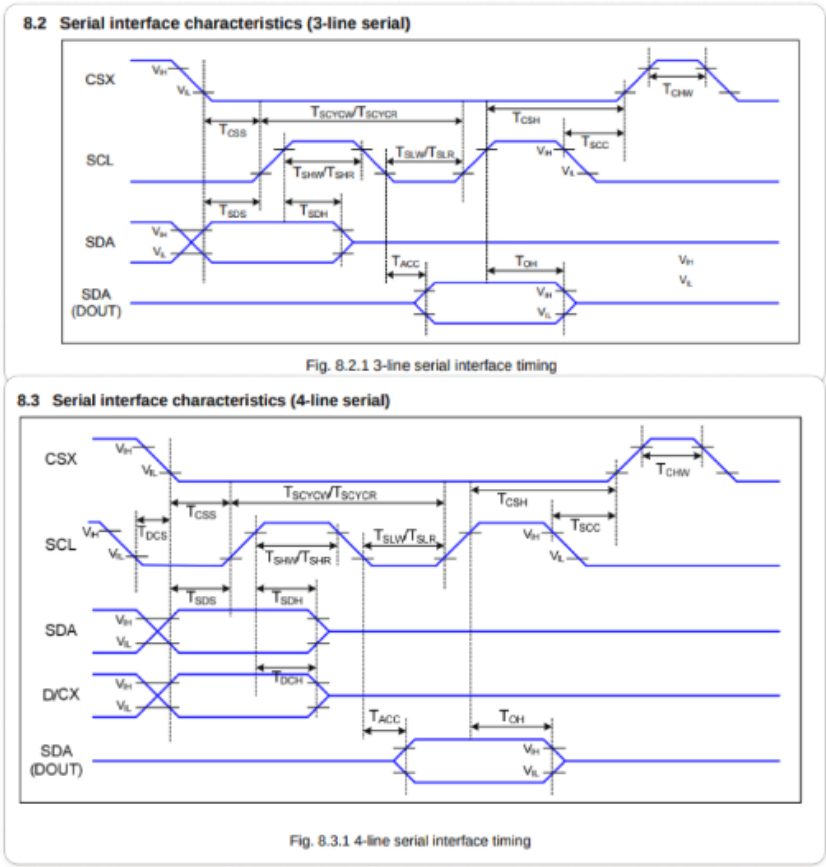


Fig. 5: Timing Constraints

3.3 Parallel 8080 MCU Interface

The 8080-series parallel interface is designed for high-throughput applications where SPI bandwidth becomes a bottleneck. It utilizes a bi-directional data bus along with control signals to accomplish asynchronous data transfers.

Depending on the hardware pin strap selection, the data bus can be configured to different bit-widths ($D[17:0]$). For standard microcontrollers, an 8-bit parallel bus is most common. The control lines involved in the 8080 parallel mode are described below:

- **CSX (Chip Select):** Enables communication when held low (`LOW`).
- **D/CX (Data/Command Select):** Determines if the transfer is a command/status or display data. Held `LOW` for commands and `HIGH` for data parameters.

Table 1: 4-line Serial Interface Characteristics

Signal	Symbol	Parameter	MIN	MAX	Unit
CSX	TCSS	Chip select setup time (write)	15		ns
	TCSH	Chip select hold time (write)	15		ns
	TCSS	Chip select setup time (read)	60		ns
	TSCC	Chip select hold time (read)	65		ns
	TCHW	Chip select "H" pulse width	40		ns
SCL	TSCYCW	Serial clock cycle (Write)	66		ns
	TSHW	SCL "H" pulse width (Write)	30		ns
	TSLW	SCL "L" pulse width (Write)	30		ns
	TSCYCR	Serial clock cycle (Read)	150		ns
	TSHR	SCL "H" pulse width (Read)	60		ns
	TSLR	SCL "L" pulse width (Read)	60		ns
D/CX	TDCS	D/CX setup time		0	ns
	TDCH	D/CX hold time	10		ns
SDA (DIN) (DOUT)	TSDS	Data setup time	10		ns
	TSDH	Data hold time	10		ns
	TACC	Access time	10	50	ns
	TOH	Output disable time		50	ns

- **WRX (Write Strobe):** A rising edge on this active-low signal latches the data from the host MCU MCU bus ($D[x:0]$) into the ST7735 internal registers.
- **RDX (Read Strobe):** A rising edge on this active-low signal drives internal display data or status registers out onto the MCU data bus.
- **D[17:0] or D[7:0] (Data Bus):** Bi-directional parallel lines used to transfer data packets.

Compared to SPI, where each pixel requires 16 sequential clock cycles on a single line, the 8-bit 8080 parallel interface can transfer a full RGB565 pixel in just two write cycles (one byte per strobe), significantly reducing CPU overhead and maximizing frame rates.

3.4 Function Description

IM2	IM1	IM0	Interface	Read Back Selection
0	-	-	3-line serial interface	Via the read instruction
1	0	0	8080 MCU 8-bit parallel	RDX strobe (8-bit read data and 8-bit read parameter)
1	0	1	8080 MCU 16-bit parallel	RDX strobe (16-bit read data and 8-bit read parameter)
1	1	0	8080 MCU 9-bit parallel	RDX strobe (9-bit read data and 8-bit read parameter)
1	1	1	8080 MCU 18-bit parallel	RDX strobe (18-bit read data and 8-bit read parameter)

Table 2: Selection of MCU Interface

IM2	IM1	IM0	Interface	RDX	WRX	D/CX	Read back selection
0	-	-	3-line serial	Note1	Note1	Note1	D[17:1]: unused D0: SDA
1	0	0	8080 8-bit parallel	RDX	WRX	D/CX	D[17:8]: unused D7-D0: 8 bit data
1	0	1	8080 16-bit parallel	RDX	WRX	D/CX	D[17:16]: unused D15-D0: 16 bit data
1	1	0	8080 9-bit parallel	RDX	WRX	D/CX	D[17:9]: unused D8-D0: 9 bit data
1	1	1	8080 18-bit parallel	RDX	WRX	D/CX	D17-D0: 18 bit data

Table 3: Selection of MCU Interface

Note1: Unused pins can be open or connected to DGND or VDDI

In this document we are going to use the **3-Line serial** (IM2,IM1,IM0 = "0xx")

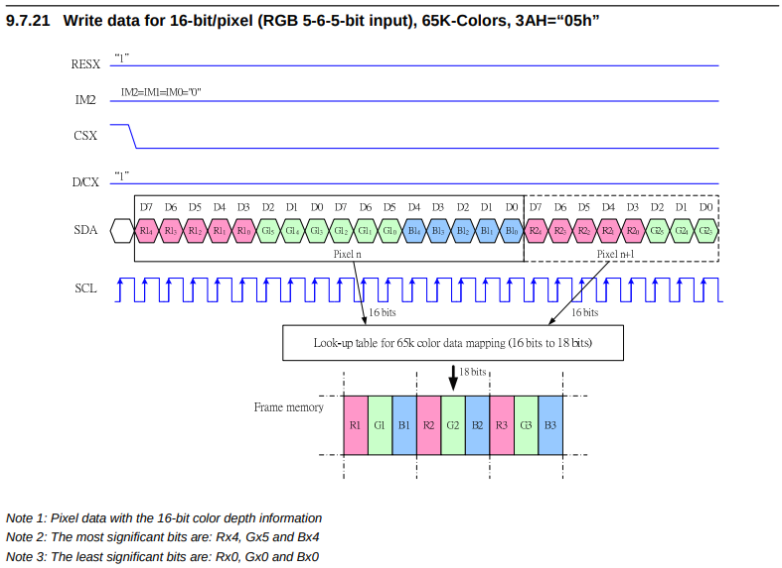


Fig. 6: Write cycle for 565

3.5 Color Frame

We could control the color frame bits by modifying the COLMOD Register. The COLMOD Register could take four possible values by loading values after 3Ah

COLMOD	Frame	RGB	Colors
011	12-bit	444	4k
101	16-bit	565	65k
110	18-bit	666	262k
111	No used	N/A	N/a

Table 4: Selection of MCU Interface

We are going to use the 565 Frame, therefore the Write data cycle would be the shown on the next figure

3.6 Data Streaming Order

There are two modes of Panel Resolution With the register (MADCTL) Memory

GM2	GM1	GM0	Panel Resolution
0	0	0	132RGB x 162
0	1	1	128RGB x 160

Table 5: Selection of MCU Interface

Data Access Control we could control multiple configurations of the screen

We could set the orientation by modifying the MV, MX, MY bits. The MV exchanges x and y axis, MX and MY flips the X and Y axis, respectively. Note that the stream begins from B (Begin) and goes to E (End)

10.1.26 MADCTL (36h): Memory Data Access Control

36H	MADCTL (Memory Data Access Control)												
Inst / Para	D/CX	WRX	RDX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX
MADCTL	0	↑	1	-	0	0	1	1	0	1	1	0	(36h)
Parameter	1	↑	1	-	MY	MX	MV	ML	RGB	MH	-	-	

-This command defines read/ write scanning direction of frame memory.

Bit	NAME	DESCRIPTION
MY	Row Address Order	These 3bits controls MCU to memory write/read direction.
MX	Column Address Order	
MV	Row/Column Exchange	
ML	Vertical Refresh Order	LCD vertical refresh direction control '0' = LCD vertical refresh Top to Bottom '1' = LCD vertical refresh Bottom to Top
RGB	RGB-BGR ORDER	Color selector switch control '0' =RGB color filter panel, '1' =BGR color filter panel)
MH	Horizontal Refresh Order	LCD horizontal refresh direction control '0' = LCD horizontal refresh Left to right '1' = LCD horizontal refresh right to left

Fig. 7: MADCTL Register

9.10.3 Frame Data Write Direction According to the MADCTL parameters (MV, MX and MY)

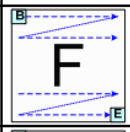
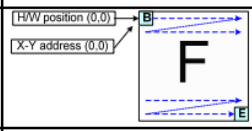
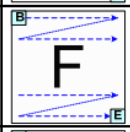
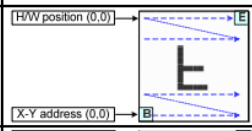
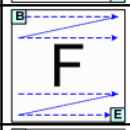
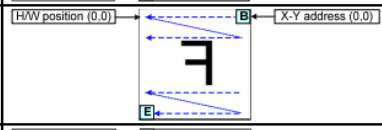
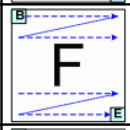
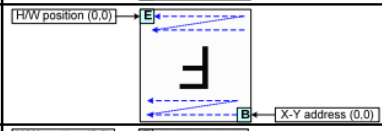
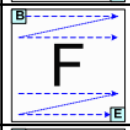
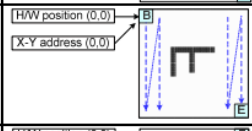
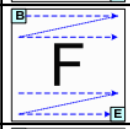
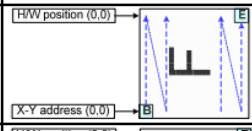
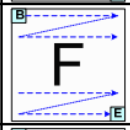
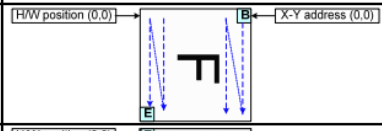
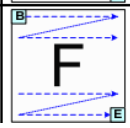
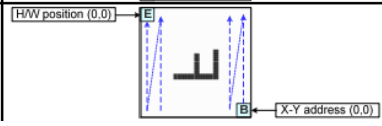
Display Data Direction	MADCTL Parameter MV	MX	MY	Image in the Host (MPU)	Image in the Driver (DDRAM)
Normal	0	0	0		
Y-Mirror	0	0	1		
X-Mirror	0	1	0		
X-Mirror Y-Mirror	0	1	1		
X-Y Exchange	1	0	0		
X-Y Exchange Y-Mirror	1	0	1		
X-Y Exchange X-Mirror	1	1	0		
X-Y Exchange X-Mirror Y-Mirror	1	1	1		

Fig. 8: Enter Caption

3.7 Gamma Adjustments (Positive and Negative Polarity)

The ST7735 controller features highly customizable Gamma Correction registers (`ST7735_GMCTRP1` for positive polarity and `ST7735_GMCTRN1` for negative polarity). In Liquid Crystal Displays (LCDs), gamma correction is not merely a software tool for contrast styling, but a critical hardware requirement to compensate for the non-linear electro-optical transfer characteristics of the liquid crystal layer.

3.7.1 Understanding Polarity in Gamma Control

Liquid crystal molecules are sensitive to the root-mean-square (RMS) value of an electric field. If a constant DC voltage is applied continuously in a single direction across the liquid crystal cells, it causes ion polarization and chemical degradation, permanently damaging the screen (a phenomenon known as *image sticking* or LCD burn-in).

To prevent this, TFT displays use an Alternating Current (AC) driving method where the polarity of the voltage applied to each pixel is inverted periodically (typically per frame or per line) relative to a common reference voltage (V_{COM}).

- **Positive Polarity (`GMCTRP1`):** Defines the voltage-to-grayscale mapping curve when the pixel voltage is higher than V_{COM}.
- **Negative Polarity (`GMCTRN1`):** Defines the voltage-to-grayscale mapping curve when the pixel voltage is lower than V_{COM}.

Because the physical behavior of liquid crystals can vary slightly depending on whether the charge is positive or negative, independent 16-argument register arrays are required to guarantee that a specific grayscale value outputs the exact same optical luminance in both polarities, preventing screen flickering.

3.7.2 Determination of Gamma Values

The 16 hexadecimal parameters passed to each gamma register block represent specific tap points along a digital-to-analog converter (DAC) voltage resistor ladder. These target points adjust key characteristics of the curve:

- **Low-order parameters** alter the black levels and dark gray gradients.
- **Mid-order parameters** fine-tune the mid-tones and linearity of the color distribution.
- **High-order parameters** determine the peak white saturation levels.

In practical driver development, these values are **empirically determined by the display module manufacturer**. The screen vendor utilizes specialized optical instrumentation (such as colorimeters and spectrophotometers) to measure the luminance response across the grayscale spectrum on a bare glass panel panel sample.

The factory engineers then calculate the exact DAC voltage offsets required to match standard target curves (typically a Gamma 2.2 profile, which matches human visual perception). The resulting magic numbers—such as the ones embedded in our `init_cmd[]` array—are hardcoded into the initialization routine to ensure accurate color reproduction, stable grayscale transitions, and identical luminance across both positive and negative driving cycles.

3.8 Registers

Finally, here we got all the registers

Table 10.1.1 System Function command List (1)

Instruction	Refer	D/CX	WRX	RDX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	Hex	Function
NOP	10.1.1	0	↑	1	-	0	0	0	0	0	0	0	0	(00h)	No Operation
SWRESET	10.1.2	0	↑	1	-	0	0	0	0	0	0	0	1	(01h)	Software reset
RDDID	10.1.3	0	↑	1	-	0	0	0	0	0	1	0	0	(04h)	Read Display ID
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	ID17	ID16	ID15	ID14	ID13	ID12	ID11	ID10		ID1 read
		1	1	↑	-	1	ID26	ID25	ID24	ID23	ID22	ID21	ID20		ID2 read
		1	1	↑	-	ID37	ID36	ID35	ID34	ID33	ID32	ID31	ID30		ID3 read
RDDST	10.1.4	0	↑	1	-	0	0	0	0	1	0	0	1	(09h)	Read Display Status
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	BSTON	MY	MX	MV	ML	RGB	MH	ST24		-
		1	1	↑	-	ST23	IFPF2	IFPF1	IFPF0	IDMON	PTLON	SLOUT	NORON		-
		1	1	↑	-	VSSON	ST14	INVON	ST12	ST11	DISON	TEON	GCS2		-
		1	1	↑	-	GCS1	GCS0	TELOM	ST4	ST3	ST2	ST1	ST0		-
RDDPM	10.1.5	0	↑	1	-	0	0	0	0	1	0	1	0	(0Ah)	Read Display Power
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	BSTON	IDMON	PTLON	SLPOUT	NORON	DISON	-	-		-
RDD MADCTL	10.1.6	0	↑	1	-	0	0	0	0	1	0	1	1	(0Bh)	Read Display
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	MY	MX	MV	ML	RGB	MH	-	-		-
RDD COLMOD	10.1.7	0	↑	1	-	0	0	0	0	1	1	0	0	(0Ch)	Read Display Pixel
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	0	0	0	0	-	IFPF2	IFPF1	IFPF0		-
RDDIM	10.1.8	0	↑	1	-	0	0	0	0	1	1	0	1	(0Dh)	Read Display Image
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	VSSON	D6	INVON	-	-	GCS2	GCS1	GCS0		-
RDDSM	10.1.9	0	↑	1	-	0	0	0	0	1	1	1	0	(0Eh)	Read Display Signal
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	TEON	TELOM	-	-	-	-	-	-		-

ST7735

Table 10.1.2 System Function command List (2)

Instruction	Refer	D/C	WR	RDX	D17-	D7	D6	D5	D4	D3	D2	D1	D0	Hex	Function
SLPIN	10.1.10	0	↑	1	-	0	0	0	1	0	0	0	0	(10h)	Sleep in & booster off
SLPOUT	10.1.11	0	↑	1	-	0	0	0	1	0	0	0	1	(11h)	Sleep out & booster on
PTLON	10.1.12	0	↑	1	-	0	0	0	1	0	0	1	0	(12h)	Partial mode on
NORON	10.1.13	0	↑	1	-	0	0	0	1	0	0	1	1	(13h)	Partial off (Normal)
INVOFF	10.1.14	0	↑	1	-	0	0	1	0	0	0	0	0	(20h)	Display inversion off
INVON	10.1.15	0	↑	1	-	0	0	1	0	0	0	0	1	(21h)	Display inversion on
GAMSET	10.1.16	0	↑	1	-	0	0	1	0	0	1	1	0	(26h)	Gamma curve select
		1	↑	1	-	-	-	-	-	GC3	GC2	GC1	GC0		-
DISPOFF	10.1.17	0	↑	1	-	0	0	1	0	1	0	0	0	(28h)	Display off
DISPON	10.1.18	0	↑	1	-	0	0	1	0	1	0	0	1	(29h)	Display on
CASET	10.1.19	0	↑	1	-	0	0	1	0	1	0	1	0	(2Ah)	Column address set
		1	↑	1	-	XS15	XS14	XS13	XS12	XS11	XS10	XS9	XS8		X address start: 0≦XS≦X
		1	↑	1	-	XS7	XS6	XS5	XS4	XS3	XS2	XS1	XS0		
		1	↑	1	-	XE15	XE14	XE13	XE12	XE11	XE10	XE9	XE8		X address end: S≦XE≦X
		1	↑	1	-	XE7	XE6	XE5	XE4	XE3	XE2	XE1	XE0		
RASET	10.1.20	0	↑	1	-	0	0	1	0	1	0	1	1	(2Bh)	Row address set
		1	↑	1	-	YS15	YS14	YS13	YS12	YS11	YS10	YS9	YS8		Y address start: 0≦YS≦Y
		1	↑	1	-	YS7	YS6	YS5	YS4	YS3	YS2	YS1	YS0		
		1	↑	1	-	YE15	YE14	YE13	YE12	YE11	YE10	YE9	YE8		Y address end:S≦YE≦Y
		1	↑	1	-	YE7	YE6	YE5	YE4	YE3	YE2	YE1	YE0		
RAMWR	10.1.21	0	↑	1	-	0	0	1	0	1	1	0	0	(2Ch)	Memory write
		1	↑	1	-	D7	D6	D5	D4	D3	D2	D1	D0		Write data
RAMRD	10.1.22	0	↑	1	-	0	0	1	0	1	1	1	0	(2Eh)	Memory read
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	D7	D6	D5	D4	D3	D2	D1	D0		Read data

Table 10.1.3 System Function command List (3)

Instruction	Refer	D/CX	WRX	RDX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	Hex	Function
PTLAR	10.1.23	0	↑	1	-	0	0	1	1	0	0	0	0	(30h)	Partial start/end address set
		1	↑	1	-	PSL15	PSL14	PSL13	PSL12	PSL11	PSL10	PSL9	PSL8		Partial start address (0,1,2, ..P)
		1	↑	1	-	PSL7	PSL6	PSL5	PSL4	PSL3	PSL2	PSL1	PSL0		
		1	↑	1	-	PEL15	PEL14	PEL13	PEL12	PEL11	PEL10	PEL9	PEL8		Partial end address (0,1,2, .., P)
		1	↑	1	-	PEL7	PEL6	PEL5	PEL4	PEL3	PEL2	PEL1	PEL0		
TEOFF	10.1.24	0	↑	1	-	0	0	1	1	0	1	0	0	(34h)	Tearing effect line off
TEON	10.1.25	0	↑	1	-	0	0	1	1	0	1	0	1	(35h)	Tearing effect mode set & on
		1	↑	1	-	-	-	-	-	-	-	-	TELOM		Mode1: TELOM="0" Mode2: TELOM="1"
MADCTL	10.1.26	0	↑	1	-	0	0	1	1	0	1	1	0	(36h)	Memory data access control
		1	↑	1	-	MY	MX	MV	ML	RGB	MH	-	-		-
IDMOFF	10.1.27	0	↑	1	-	0	0	1	1	1	0	0	0	(38h)	Idle mode off
IDMON	10.1.28	0	↑	1	-	0	0	1	1	1	0	0	1	(39h)	Idle mode on
COLMOD	10.1.29	0	↑	1	-	0	0	1	1	1	0	1	0	(3Ah)	Interface pixel format
		1	↑	1	-	-	-	-	-	-	IFPF2	IFPF1	IFPF0		Interface format
RDID1	10.1.30	0	↑	1	-	1	1	0	1	1	0	1	0	(DAh)	Read ID1
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	ID17	ID16	ID15	ID14	ID13	ID12	ID11	ID10		Read parameter
RDID2	10.1.31	0	↑	1	-	1	1	0	1	1	0	1	1	(DBh)	Read ID2
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	1	ID26	ID25	ID24	ID23	ID22	ID21	ID20		Read parameter
RDID3	10.1.32	0	↑	1	-	1	1	0	1	1	1	0	0	(DCh)	Read ID3
		1	1	↑	-	-	-	-	-	-	-	-	-		Dummy read
		1	1	↑	-	ID37	ID36	ID35	ID34	ID33	ID32	ID31	ID30		Read parameter

"-": Don't care

Note 1: After the H/W reset by RESX pin or S/W reset by SWRESET command, each internal register becomes default state (Refer "RESET TABLE" section)

Note 2: Undefined commands are treated as NOP (00 h) command.

Note 3: B0 to D9 and DA to F are for factory use of driver supplier.

Note 4: Commands 10h, 12h, 13h, 20h, 21h, 26h, 28h, 29h, 30h, 36h (ML parameter only), 38h and 39h are updated during V-sync when Module is in Sleep Out Mode to avoid abnormal visual effects. During Sleep In mode, these commands are updated immediately. Read status (09h), Read Display Power Mode (0Ah), Read Display MADCTL (0Bh), Read Display Pixel Format (0Ch), Read Display Image Mode (0Dh), Read Display Signal Mode (0Eh).

3.9 Initialization Sequence

The following array contains the configuration sequence for the ST7735R controller. To distinguish between commands and data control packets within the bare-metal driver, a `0x0100` prefix mask is applied to metadata parameters, while direct commands maintain their hardware definitions.

```
uint16_t init_cmd[] = {
    ST7735_SWRESET, // 1: Software reset, 0 args, w/delay
    ST7735_SLPOUT , // 2: Out of sleep mode, 0 args, w/delay
    ST7735_FRMCTR1, // 3: Frame rate ctrl - normal mode, 3 args:
    0x0101,         //    Arg 1: RTNA setting
    0x012C,         //    Arg 2: FPA setting
    0x012D,         //    Arg 3: BPA setting. Rate = fosc/(1x2
        +40) * (LINE+2C+2D)
    ST7735_FRMCTR2, // 4: Frame rate control - idle mode, 3 args:
    0x0101,         //    Arg 1: RTNB setting
    0x012C,         //    Arg 2: FPB setting
    0x012D,         //    Arg 3: BPB setting. Rate = fosc/(1x2
        +40) * (LINE+2C+2D)
    ST7735_FRMCTR3, // 5: Frame rate ctrl - partial mode, 6 args:
    0x0101,         //    Arg 1: RTNC setting
    0x012C,         //    Arg 2: FPC setting
    0x012D,         //    Arg 3: BPC setting (Dot inversion mode)
    0x0101,         //    Arg 4: RTNC setting
    0x012C,         //    Arg 5: FPC setting
    0x012D,         //    Arg 6: BPC setting (Line inversion mode
        )
```

```
    ST7735_INVCTR , // 6: Display inversion ctrl, 1 arg, no delay:
    0x0107,         //    Arg 1: No inversion (Line inversion on full/idle/partial)
    ST7735_PWCTR1 , // 7: Power control, 3 args, no delay:
    0x01A2,         //    Arg 1: GVDD setup
    0x0102,         //    Arg 2: VCI1 / VGH / VGL setup (-4.6V)
    0x0184,         //    Arg 3: Opamp bias selection (AUTO mode)
    ST7735_PWCTR2 , // 8: Power control, 1 arg, no delay:
    0x01C5,         //    Arg 1: VGH25 = 2.4C, VGSEL = -10, VGH = 3 * AVDD
    ST7735_PWCTR3 , // 9: Power control, 2 args, no delay:
    0x010A,         //    Arg 1: Opamp current small (Normal mode)
    0x0100,         //    Arg 2: Boost frequency (Normal mode)
    ST7735_PWCTR4 , // 10: Power control, 2 args, no delay:
    0x018A,         //    Arg 1: BCLK/2, Opamp current small & Medium low (Idle)
    0x012A,         //    Arg 2: Boost frequency (Idle mode)
    ST7735_PWCTR5 , // 11: Power control, 2 args, no delay:
    0x018A,         //    Arg 1: Opamp current and boost (Partial mode)
    0x01EE,         //    Arg 2: Boost frequency (Partial mode)
    ST7735_VMCTR1 , // 12: Power control, 1 arg, no delay:
    0x010E,         //    Arg 1: VCOM voltage setting
    ST7735_INVOFF , // 13: Don't invert display, no args, no delay
    ST7735_MADCTL , // 14: Memory access control (directions), 1 arg:
    ST7735_ROTATION,//    Arg 1: Row/Col address scan, bottom to top refresh
    ST7735_COLMOD , // 15: Set color mode, 1 arg, no delay:
    0x0105 ,        //    Arg 1: 16-bit color format (RGB565)
    ST7735_CASET  , // 16: Column addr set, 4 args, no delay:
    0x0100,         //    Arg 1: XSTART high byte
    0x0100,         //    Arg 2: XSTART low byte = 0
    0x0100,         //    Arg 3: XEND high byte
    0x017F,         //    Arg 4: XEND low byte = 127
    ST7735_RASET  , // 17: Row addr set, 4 args, no delay:
    0x0100,         //    Arg 1: YSTART high byte
    0x0100,         //    Arg 2: YSTART low byte = 0
    0x0100,         //    Arg 3: YEND high byte
    0x017F,         //    Arg 4: YEND low byte = 127
    ST7735_GMCTRP1, // 18: Gamma Adj (+ pol), 16 args, no delay:
    0x0102, 0x011c, 0x0107, 0x0112, // Args 1 to 4: Curve map
    0x0137, 0x0132, 0x0129, 0x012d, // Args 5 to 8: Curve map
    0x0129, 0x0125, 0x012B, 0x0139, // Args 9 to 12: Curve map
    0x0100, 0x0101, 0x0103, 0x0110, // Args 13 to 16: Curve map
    ST7735_GMCTRN1, // 19: Gamma Adj (- pol), 16 args, no delay:
    0x0103, 0x011d, 0x0107, 0x0106, // Args 1 to 4: Curve map
    0x012E, 0x012C, 0x0129, 0x012D, // Args 5 to 8: Curve map
    0x012E, 0x012E, 0x0137, 0x013F, // Args 9 to 12: Curve map
    0x0100, 0x0100, 0x0102, 0x0110, // Args 13 to 16: Curve map
    ST7735_NORON  , // 20: Normal display on, no args, w/delay
    ST7735_DISPON   // 21: Main screen turn on, no args w/delay
};
```

4 XPT2046 Touch Controller

For the touch functionality, the XPT2046 controller is used. It communicates with the host microcontroller via SPI and provides accurate touch position data. The controller supports 12-bit resolution for both X and Y coordinates, allowing for precise touch detection.

4.1 Control Byte

We could request the touch position by sending a control byte to the XPT2046. The control byte format is as follows: With changing A2-A0 we could request the X or Y

BIT7(MSB)	BIT 6	BIT 5	BIT 4	BIT 3	BIT2	BIT 1	BIT 0(LSB)
S	A2	A1	A0	MODE	SER/$\overline{\text{DFR}}$	PD1	PD0

Table 6. Order of the Control Bits in the Control Byte

BIT	NAME	DESCRIPTION
7	S	Start bit. Control byte starts with first high bit on DIN.A new control byte can start every 15th clock cycle in 12-bit conversion mode or every 11th clock cycle in 8-bit conversion mode (see Figure 16).
6-4	A2-A0	Channel Select bits. Along with the SER/DFR bit,these bits control the setting of the multiplexer input,touch driver switches, and reference inputs (seeTable 1 and Figure 16).
3	MODE	12-Bit/8-Bit Conversion Select bit. This bit controls the number of bits for the next conversion: 12-bits(low) or 8-bits (high).
2	SER/$\overline{\text{DFR}}$	Single-Ended/Differential Reference Select bit. Along with bits A2-A0, this bit controls the setting of the multiplexer input, touch driver switches, and reference inputs (see Table 1 and Table 2).
1-0	PD1-PD0	Power-Down Mode Select bits. Refer to Table 5 fordetails.

Table 7. Descriptions of the Control Bits within the Control Byte

Fig. 9: XPT2046

position, with the followiing table we could see what we could request with the control byte

$$\begin{cases} Y - Position: & [A2:A0] = 001 \\ X - Position: & [A2:A0] = 101 \\ Z1 - Position: & [A2:A0] = 011 \\ Z2 - Position: & [A2:A0] = 100 \end{cases}$$

We also could set the PD0 and PD1 bits to set the power down mode, with the following table we could see the different modes. In this case we are going to use the mode 00, which is the power down mode with internal reference off, and the internal reference is used for the conversion. There is also a bit called SER/DFR, which is

A2	A1	A0	+REF	−REF	YN	XP	YP	Y-POSITION	X-POSITION	Z_1-POSITION	Z_2-POSITION	DRIVERS
0	0	1	YP	YN		+IN		M				YP, YN
0	1	1	YP	XN		+IN				M		YP,
1	0	0	YP	XN	+IN						M	YP,
1	0	1	XP	XN			+IN		M			XP,

Fig. 10: Input Configuration (DIN), Differential Reference Mode (SER/DFR low)

PD1	PD0	$\overline{\text{PENIRQ}}$	DESCRIPTION
0	0	Enabled	Power-Down Between Conversions. When each conversion is finished, the converter enters a low-power mode. At the start of the next conversion, the device instantly powers up to full power. There is no need for additional delays to ensure full operation, and the very first conversion is valid. The Y− switch is on when in power-down.
0	1	Disabled	Reference is off and ADC is on.
1	0	Enabled	Reference is on and ADC is off.
1	1	Disabled	Device is always powered. Reference is on and ADC is on.

Fig. 11: Power-Down and Internal Reference Selection

used to set the differential or single-ended reference mode, in this case we are going to use the differential reference mode, which is the most accurate mode. So finally we are going to build our control byte with the following configuration:

- Mode = 0 (12-bit resolution)
- SER/$\overline{\text{DFR}}$ = 0 (Differential Reference Mode)
- PD1 = 0 (Power Down Mode with Internal Reference Off)
- PD0 = 0 (Power Down Mode with Internal Reference Off)

Table 6: Control Byte

S	A2	A1	A0	MODE	SER/DFR	PD1	PD0	Hex	Request
1	0	0	1	0	0	0	0	0x90	Y-Position
1	1	0	1	0	0	0	0	0xD0	X-Position
1	0	1	1	0	0	0	0	0xB0	Z1-Position
1	1	0	0	0	0	0	0	0xC0	Z2-Position

5 Description

The library for the ST7735-based TFT display provides a simple and efficient interface for handling graphics through the SPI communication protocol.

This library abstracts the complexity of the command sequences and internal operation of the ST7735, offering intuitive functions to initialize the display, print color rectangles,fill the creen with one color, draw pixels, and test the colors on the screen, each one integrated on a 4 simple function to use:

Screen1.FillScreen(color).

Screen1.FillRectangle(posx, posy, width, height, color).

Screen1.DrawPixel(posyx, posy, color).

test(void);

6 Purpose

This library aims to support rapid development of visual interfaces, diagnostic screens, and real-time graphical outputs, offering a set of functions optimized for performance and compatibility with a wide range of microcontrollers. It is designed to serve both beginners and advanced users who require a practical and flexible tool to integrate TFT graphics into their projects.

7 How to use properly

This section explains how to integrate and use the ST7735 TFT driver and the oscilloscope demo on an STM32F446 MCU.

7.1 Driver Initialization

1. Instantiate the driver object (constructor calls low-level `config()` for RCC/GPIO/SPI).
2. Call `INIT_FN()` once after reset to send the ST7735 init sequence.
3. Clear or paint the screen using `FillScreen(color)`.

Example:

```
TFT_ST7735 tft;
tft.INIT_FN();
tft.FillScreen(COLOR_BLACK);
```

7.2 Drawing Primitives

- Pixel: `DrawPixel(x, y, color)`
- Rectangle: `FillRectangle(x, y, w, h, color)` (auto clipping)
- Full screen: `FillScreen(color)`
- Text: `WriteChar(x, y, ch, font, fg, bg)` and `WriteString(x, y, str, font, fg, bg)`

Colors use RGB565; predefined constants (`COLOR_RED`, `COLOR_BLUE`, etc.) live in the header.

7.3 Oscilloscope Demo Flow

Provided in `main_osc.cpp`:

1. Configure PLL and system clock via `conf_osc()`.
2. Enable and start continuous conversion on ADC1 channel at `PA0`.
3. Initialize TFT and draw axes using rectangles.
4. In loop: read `ADC1->DR`, scale value (`>>5`) to horizontal pixel range, increment vertical time index, plot with `DrawPixel()`.
5. When vertical index reaches bottom, clear plot region and restart for a scrolling effect.

7.4 Build Notes

The code uses bare-metal register access (no HAL). Ensure your linker script and startup file match STM32F446. SPI baud is set to fPCLK/16. If the display shows nothing, verify wiring and that HSE/PLL configuration succeeds.

7.5 Testing

1. Power the board and TFT (3.3V only).
2. Flash firmware containing either `main_tst.cpp` (color test) or `main_osc.cpp` (oscilloscope).
3. For oscilloscope: vary the analog input on PA0 (potentiometer or waveform source) and observe horizontal deflection of the cyan trace.

8 Diagram

In the figure 12 the test circuit with the test code printing multiple rectangles, notice that the connections are in the table 8.

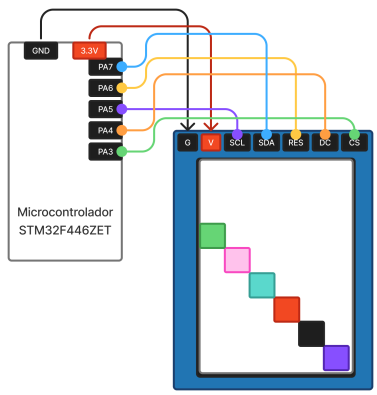


Fig. 12: Schematic

TFT Pin	STM32F446 Pin	Purpose
VCC	3.3V	Power
GND	GND	Ground
SCK	PA5	SPI1 SCK (AF5)
MOSI	PA7	SPI1 MOSI (AF5)
CS	PA3	Chip Select (GPIO)
DC	PA4	Data/Command select (GPIO)
RST	PA6	Hardware reset (GPIO)

Optional: connect a variable analog signal (e.g. potentiometer wiper) to `PA0` for the oscilloscope demo (ADC input).

9 Limitations and Enhancements

9.1 Current Limitations

- **Polling SPI**: All transfers busy-wait; CPU cannot perform other tasks concurrently.
- **No DMA**: Large area fills and waveform plotting are slower than possible; higher CPU usage.
- **Fixed Rotation**: Only one memory access orientation initialized; no runtime rotation switching.
- **Limited Primitives**: Lines, circles, bitmaps, and sprites are not yet implemented.
- **No Text Clipping**: Long strings may draw off-screen without wrapping or truncation.
- **Oscilloscope Scaling**: Simple right-shift scaling (`>>5`) reduces resolution; no trigger or persistence.
- **Blocking Clear**: Screen clear for oscilloscope resets full plot; no partial scrolling buffer.

9.2 Potential Enhancements

- **SPI DMA + Interrupts**: Offload pixel bursts; increase frame throughput.
- **Drawing Toolkit**: Add line (Bresenham), circle (Midpoint), bitmap blitting, and gradient fills.
- **Runtime Rotation**: Implement MADCTL updates with offset correction and cached clipping.
- **Text System**: Clipping region, wrapping, proportional fonts, and transparency support.
- **Waveform Features**: Trigger modes (edge/level), adjustable vertical/horizontal scaling, persistence with ring buffer.
- **Double Buffering**: Compose frames off-screen then push for flicker-free updates.
- **UI Layer**: Basic widgets (labels, value bars, status icons) for sensor dashboards.
- **Color/Theme Profiles**: Map amplitude ranges to color gradients (heatmap visualization).

9.3 Recommended Next Steps

Prioritize DMA integration and trigger logic for oscilloscope, then expand primitive set and text handling to support richer UI applications (data logger, multi-channel monitor).

- Configure SPI in master (one board) and slave (other board) mode.
- Configure UART on both boards for terminal output text (Putty etc).
- Define and use two control commands over SPI: start and finish.
- Implement the count logic (0-9) on the slave, and delay/loop logic on the master.
- Print meaningful messages via UART and maintain a cycle counter on the slave